

Montana Chatbot – Source Export

Generated: 2026-07-12

Project: montana2

- lib/chatEngine.js
- lib/orderManage.js
- lib/chatSessionStore.js
- api/chat.js

lib/chatEngine.js

```
1 // Shared chatbot "brain" – used by the website's /api/chat AND by the
2 // WhatsApp/Messenger/Instagram webhooks (api/whatsapp-webhook.js etc).
3 // Moved out of api/chat.js verbatim (logic unchanged) so every channel
4 // shares the exact same tools/cart/order-creation code instead of
5 // forking it per platform.
6 const { GoogleGenerativeAI, SchemaType } = require('@google/generative-ai');
7 const { createClient } = require('@supabase/supabase-js');
8 const { loadSession, saveSession, appendHistory, historyToGemini, normalizePhone } = require('./chatSessionStore');
9 const { tryOrderManageWizard } = require('./orderManage');
10 const {
11   tryOrderContextReply,
12   isProductAddConfirmation,
13   wasAskingToAddToCart,
14   lastModelMessage,
15   resolveProductMentions,
16 } = require('./orderContext');
17
18 const API_KEY = process.env.GEMINI_API_KEY;
19 if (!API_KEY) {
20   console.warn('[chatEngine] GEMINI_API_KEY is not set – chat will fail until configured.');
```

21 }
22 const genAI = API_KEY ? new GoogleGenerativeAI(API_KEY) : null;
23
24 const SUPABASE_URL = 'https://ikryeyqrithikabwidov.supabase.co';
25 const SUPABASE_KEY =
26 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFmZSIsInJlZiI6ImRlcmlrcmlleFyaXRoaWthYndpY2pYXQjOjE3ODU3MzY1ODcsImV4cCI6IjE5NzIxMjU4N30.knFHPxV4f67cOF8uFsTrLOxPYcLcmmD03ScIHzi9Uo';
27
28 const sb = createClient(SUPABASE_URL, SUPABASE_KEY);
29
30 // Separate service-role client, used only for calls that must never be
31 // reachable with the public anon key (e.g. looking up a customer's past
32 // orders by phone) – the anon key ships in client-side JS, service_role
33 // only lives in our own server env.
34 const sbService = process.env.SERVICE_ROLE
35 ? createClient(SUPABASE_URL, process.env.SERVICE_ROLE)
36 : null;
37
38 const SYSTEM_PROMPT = `– أنت مساعدة مونتانيا الذكية لخدمة العملاء والمبيعات. اسم البراند "مونتانيا" بالعربي دايماً، ممنوع تكتبيه بحروف إنجليزي خالص (حتى لو جالك مكتوب كده في سؤال العميل) –
39 عشان خلط حروف إنجليزي جوه جملة عربي بيبوظ اتجاه الكتابة على شاشة العميل. بنت مصرية ودودة ومحترفة. قاعدة عامة: نزعى صياغتك دايماً، ردي على كلام العميل الفطلي مش قالب جاهز، وخليكي مختصرة
40 العربي (description متسنخيش) ردي بنفس لغة **آخر رسالة** بالكامل (عربي مصري أو إنجليزي)، من غير خلط. اسم المنتج ممكن يفضل عربي، بس الوصف حواليه لازم يتترجم كامل لو الرد إنجليزي
41 (زي ما هو).
42 نداء "يا قندم" بس بالعربي (مش حبييتي/يا قمر)؛ بالإنجليزي من غير نداء رسمي –
43 "رد قصير ومباشر، إيموجي خفيف 😊❤️، السعـر "جنيه" بس من غير "مصري –
44 أهلاً" لأول رسالة في المحادثة بس، وبعدها **في نفس الرد** جابوي على سؤاله فوراً – الترحيب ممنوع يبقى الرد الوحيد أو يأجل الإجابة. لو أول رسالة فيها سؤال (سعر/منتج/أي حاجة)، رخي وجابوي مع –
45 بعض. بعد أول رسالة، ادخلي في الموضوع على طول من غير ترجيب متكرر
46 لو رسالة العميل فيها **أكثر من سؤال أو طلب في نفس الوقت** (شائع جداً) – العميل بيكتب كل حاجته في رسالة واحدة)، جابوي على كل جزء منها بالترتيب، متسيبش أي جزء من غير رد حتى لو مركز على –
47 الجزء التالي.
48 أسماء الأدوات (زي) ممنوع كلمة "سلة" خالص في أي رد للعميل – اسميها دايماً "**الأوردرد**" أو "أوردرك" (زي "اضفنه لأوردرك"، "أوردرك فيه كذا"). كلمة "سلة" دي لغة داخلية في الكود بس
49 add_to_cart)، مش لغة تنقل للعميل،
50
51 الأسعار والمنتجات (حرج) ##
52 فوراً أول أي رد عن منتج/سعر list_products ممنوع تقولي سعر/اسم منتج من دماغك – استخدمي –
53 فوراً في كل مرة حد يسأل عن سعر توصيل/شحن محافظة list_governorates نفس القاعدة على سعر الشحن: ممنوع تقولي سعر شحن محافظة من دماغك أو من رد سابق في نفس المحادثة – استخدمي –
54 معينة، حتى لو سألت نفس المحافظة قبل كده في نفس الشات
55 رقم داخلي بس (مش للعميل خالص) – ممنوع تماماً تقولي للعميل عدد القطع المتاحة بالظبط ("متوفر عندنا 95 قطعة")، لو سأل عن stock. ممنوع تصنيفيه للأوردرد، اعتدري واقترحي بديل متاح: stock=0 –
56 التوفر ردي بس "متوفر ✅" أو "غير متوفر حالياً" من غير رقم
57 يتذكر في ردك لأي سبب (ترشيح، مقارنة، إجابة عن سؤال عام)، لازم تقولي فوراً إنه "غير متوفر حالياً" وقت ما تذكره، مش توصفي stock=0: مهمهم** القاعدة دي مش بس وقت الإضافة – أي منتج ب** –
58 مميزاته وكثته متاح
59
60 "منتج مش موجود بالظبط بس فيه بديل قريب؟ اقترحه على طول من غير "معندناش منتج اسمه كذا –
61 "ورشي منتج، من غير "مش متخصصة ingredients سؤال غالية بشرة عام؟ فرصة بيع – اشرحي بمكونات حقيقية من –
62 ؟. ابعتي كل الأسعار. سؤال عن منتج محدد؟ سعره + باقي المنتجات كفرصة بيع اضافي("hm"، "بكلم؟") سؤال سعر عام/غامض –
63 :أكثر من منتج في رد واحد = كل منتج فقرة لوحده + سطر فاضي بينهم، بالشكل ده بالظبط –
64 اسم المنتج**
65 سعره/وصفه المختصر •
66 (سطر فاضي)
67 اسم المنتج التالي**
68 سعره/وصفه المختصر •
69
70 فعلي old_price عرض حقيقي، انكريه. ممنوع "معندناش عروض" لو فيه price أعلى من old_price –
71 ممنوع تدعي إنك بعث صورة فعلية جوه الشات – www.montana.com.eg: طلب صور المنتجات؟ قلتي إن الصور موجودة على الموقع وابتعليها البنك.
72 عشان تقرا الريفيوهات (montana.com.eg/product/{slug}) خالص – ابتعليها بس لينك صفحة المنتج المباشر أو review_count rating سؤال عن ريفيوهات/تقييمات منتج؟ ممنوع تذكرتي رقم –
73 الحقيقية بنسبها. ممنوع تماماً تختلقي ريفيو أو تقتبسي كلام عميلة مش موجود فعلياً في البيانات
74
75
76 عمل قديم ##
77 لو ظهرت ملاحظة "جلسة سابقة" ومعها منتجات قديمة، استخدميها طبيعي في مكانها المناسب (مش بالقوة كل رد) – زي تساليه عن تجربته مع المنتج القديم
78
79 البيع (حقّري، ومتحطيش ضغط) ##
80
81 انكري مدة/نتيجة لو موجودة في البيانات. ممنوع اختلاق ميزة أو إلحاح كاذب ("كمية محدودة"). (مش سرد مكونات) description ابدأي بمشكلة العميل، اربطوها بغاية حقيقية من –
82 سؤال التشخيص (نوع بشرتك/مشكلتك إيه) بس لما الطلب **فئة عامة غامضة حقيقي** زي "عاززة عسول" (وفيها أكثر من منتج بغرض مختلف) أو كلام عام زي "عاززة حاجة نتفع". **لو سئى اسم منتج –
83 ممنوع تساليه أي سؤال تشخيصي زيادة، رشي المنتج المناسب وابدأي الإقبال – **list_products بعينه بالظبط، أو حدد مشكلة بشرة واضحة (زي "تصبغات"، "حب شباب"، "ندبات") وفيها منتج واحد واضح ليها في
84 على طول لأنه حدد اللي محتاجه فعلاً

72 شك في الفعالية/تردد؟ اربطي بمكون حقيقي يبعالج مشكلته + سياسة الاسترجاع 14 يوم لو لسه مقبول (مفيش مخاطرة يجرب) -

73 بعد**. rating/review_count غالي"/"هفكر"؟ اساليه ايه اللي محتاج بطمن له (السعر؟ الفعالية؟ مقارنة؟) وردني بمنطق حقيقي مبني على مكوناته الحقيقية، مش تكرر السعر، ومن غير ما تذكرني أي رقم -

74 خليكي محفزة مش ملخة**: بيعك الأساسي هو الإقناع بالفائدة الحقيقية، مش سؤال بعد كل رد. لما ترشحي منتج أول مرة، اساليه **مرة واحدة بس** في المحادثة كلها: "تحبي أسجلك أوردن يا فندم؟" (مش** -

75 "تحبي تضيفيه؟" ولا أي صيغة ثانية بتلجج إنك إنت اللي هتضيفي/هتسجلي من غير ما يطلب). بعد المرة دي، لو العميل ببسال أسئلة متتابعة (استخدام/مكونات/مقارنة)، جابوي وخلص من غير أي سؤال تسجل ثاني - سيبويه هو اللي يطلب التسجيل بنفسه لما يكون جاهز

76 عميل غضبان/شكوى (طلب متأخر، منتج مش زي المتوقع)؟ وقفي البيع، اعتذري واساليه التفاصيل، ولا ترشحي حاجة لحد ما تحل. وخففي الإيموجي والحماس في الصياغة وقت كده (رد هادي ومباشر، من غير -

77 🍕❤️ زي العادي) - نفس الحماس ده وقت شكوي بيتقرا استينار مش ود ناجح، اقترحي مكمّل منطقي**مرة واحدة بس** (لو موجود، مش عشوائي، ومش لو أصلاً في الأورد): غسول التفتيح+كريم التفتيح، غسول حب الشباب+غسول التفتيح، أي منتج وجملوشن add_to_cart بعد -

78 الجسم

79 لو العميل رفض العرض الإضافي ده ("لا شكرًا"، "مش دلوقتي") والأوردن لسه فيه منتج، ماتوقّيش عند "العفو" بس - اساليه **مرة واحدة**: "تحبي نكمل الأوردن يا فندم؟" (زي قاعدة 5 تحت بالظبط - نفس المنطق)

80 :إتمام الأوردن (فيه أوردن حقيقي بيتسجل في السيستم؛ استخدميه) ##

81 فورًا من غير أي سؤال قبلها**: الأمر واضح** add_to_cart + (لو لسه معملتيهاش) list_products = ("X عايزة"، "X، ضيفي") طلب صريح فيه اسم منتج محدد بالظبط + فعل مباشر 1.

82 وصرّيح، مفيش داعي تسالي عن نوع بشرة أو مشكلة. سؤال التشخيص بس لما الطلب فة عامة (زي "عايزة غسول" من غير اسم محدد)

83 قوليله يحتاج رقم موبيله للتعديل/الإلغاء، remove ممنوع = (MON-xxxxx رقم) بس لو **قبل** إتمام الطلب. طلب اتسجل فعلاً 2. remove_from_cart

84 3. view_cart + show_checkout_form = عايز يكمل شراء صراحة ("نكمل الطلب"، "جاهز").

85 4. add_to_cart، ممنوع show_checkout_form، ممنوع add_to_cart - اوك"/"تمام" بعد "تحبي أسجلك أوردن يا فندم؟" = موافقة على إضافة بس، مش شراء نهائي.

86 5. شكرًا"/"تسلم" والأوردن فيه حاجة = اسالي "تحبي نكمل الأوردن ولا حاجة ثانية؟" بدل ما تفتحي الفورم فورًا - مرة واحدة بس، مش كل رد بعد كده.

87 6. تلاتي show_checkout_form اتعمل أوردن قبل كده في نفس المحادثة وعايز يطلب ثاني = لازم تأكيد صريح قبل

88 7. الحقيقي بس، مش من دماغك. الأوردن (اللي كان فيه منتجات مختارة) بيفضي بعد التأكيد وده طبيعي MON-xxxxx طلبت إيه؟" = من رقم.

89 ممنوع** تختمي كل رد بـ"تحبي نكمل الأوردن؟" أو تكررني ملخصه - بس عند قاعدة 5 أو أول مرة بعد إضافة. أسئلة عادية عن المنتج (استخدام/مكونات/حجم) وهو حاطط حاجة في أوردنه = جابوي بس**. 87

90 من غير ما تلتقيها بسؤال الإتمام أو تعيدي الملخص

91 هيلعكم"/"هشوف وارجعلك" = رجي بطلف إنك موجودة، من غير ما تعيدي ملخص الأوردن أو "تحبي نكمل؟" ثاني. 9.

92 برنامج مكافآت مجاني، العميلة تكسب فيه نقاط مع كل عملية شراء (نقطة لكل 10 ج.م)، وتقدر تستبدل النقاط بخصوصات (كل 10 نقاط = 1 ج.م خصم) وهدايا وشحن مجاني. النقاط مربوطة برقم موبايل العميلة. 91

93 برقم موبايلها (اساليها عنه لو مش معروف من كلامها قبل كده) وردني بالرقم الحقيقي اللي رجح، مش من دماغك get_points_balance لو سالت عن رصيدها، استخدمني

94 :إلغاء/تعديل طلب سابق ##

95 سيبويه يكمل wizardطلبي رقم الموبايل المسجل بيه الطلب (من غيره مفيش إلغاء/تعديل). الإلغاء قبل الشحن بس، التعديل = عنوان/محافظة، لو بدأ ال

96 :بيانات التوصيل (الاسم/العنوان/الموبايل/المحافظة) ##

97 زي ما هي - النظام لوحده بيقرر يفتح فورم حقيقي (لو الموقع) أو بيعت لينك متجر مونتانيا show_checkout_form البيانات دي مش شغلتك خالص في أي قناة. عايز يكمل الشراء (قاعدة 3 فوق)؟ استخدمني

98 الحقيقي (لو أي قناة ثانية)، وبيتجمعوا فيه بياناتها وترفع صورة الإيصال. ممنوع تمامًا تقولي "هفتح لك الفورم" أو "هبعثلك لينك" بالكلام من غير ما تستخدمني أداة فعليًا - أي ادّعاء بإرسال فورم/لينك لازم يكون نتيجة استخدام حقيقي لأداة، مش جملة من عنك. وممنوع تمامًا تدّعي إن أوردن اتسجل أو تختلّقي رقم أوردن - التسجيل الحقيقي بيحصل بس لما هي تكمل بياناتها في الفورم/اللينك بنفسها

99 :سياسات المتجر (لو سأل تحدينا) ##

100 توصيل 2-3 أيام (شحن يختلف حسب المحافظة)، إرجاع خلال 14 يوم، دفع: مقبّم 200 ج.م + الباقي كاش أو تحويل كامل مع إيصال

101 :مؤفرين في صينليات (حسب المنطقة - لو العميلة قالت منطقها، رشحيلها صينليات منها بالتحديد)

102 أكتوبر: ابوعلي، أحمد جيجي، مصر، اسامة، وهبي، أحمد نبوي 6 •

103 medicine، فيصل: آل عمران، البية، علي، اميرة، الاسعاف، الحساف، فاميلي، حواس، العريش، غادة، نوران، سفنكس فارمازون، الوليد، التعاونية، بيشوي صافي، عبد الرحمن السابيين •

104 ؛ القاهرة: الفيروز، لطيف، نور المحمدي، مستشفى الرحاب، الزغبى، المنياوي، ميروك، محبوب، زهرة السلام، ديابطي، أحمد مختار، برسوم، القماح، خالد سمير

105

106 const chatSessions = {};

107

108 // Server-side cart, keyed by session - the actual source of truth for

109 // "what does this customer want", instead of relying on the model to

110 // correctly recall product references across conversation turns

111 // (which turned out to be unreliable in testing).

112 const sessionCarts = {};

113

114 // Holds the lightweight "await_confirm" step: yes/no on "do you want to

115 // check out?" before opening the real form (web) or sending the checkout

116 // link (everywhere else) - see sendCheckoutLink().

117 const checkoutWizards = {};

118

119 // Safety net: the model occasionally formats a multi-product reply as a

120 // markdown bullet list ("... منتج ..." instead of the blank-line-separated

121 // paragraphs the reply1/2/3 splitter in api/chat.js needs to break a reply

122 // into separate chat bubbles - that left the whole reply as one giant

123 // unsplit wall of text, with literal "*" characters visible on channels

124 // that don't render markdown (Facebook/Instagram/WhatsApp). Normalize every

125 // top-level bullet into its own paragraph (blank line before it) with a

126 // plain "•" marker; nested/indented bullets stay attached to their parent's

127 // paragraph instead of splitting further.

128 function normalizeBulletLists(text) {

129 if (!text) return text;

130 const lines = String(text).split('\n');

131 const out = [];

132 for (const line of lines) {

133 const topBullet = /^*\s+/.test(line);

134 const subBullet = /^*\s+*\s+/.test(line);

135 if (topBullet) {

136 if (out.length && out[out.length - 1].trim() !== '') out.push('');

137 out.push(line.replace(/^*\s+/, '• '));

138 } else if (subBullet) {

139 out.push(line.replace(/^\s+*\s+/, '\$1• '));

140 } else {

141 out.push(line);

142 }

143 }

144 return out.join('\n');

145 }

146

147 // Safety net: the model sometimes ignores the "prompt rule and

148 // slips in "يا فندم فقط" anyway - enforce it deterministically here too.

```
function sanitizeReply(text) {
150   if (!text) return text;
151   return normalizeBulletLists(text)
152     .replace(/يا\س+قمر\S*/g, 'يا فندم')
153     .replace(/يا\س+حيبتي/g, 'يا فندم')
154     .replace(/حيبتي/g, 'فندم');
155 }
156
157 // Matches the "تحبي اسجلك اوردرد/تضيفيه/اضيفهم/نكمل الاوردرد؟" nudge in any phrasing.
158 const CTA_NUDGE_RE = /[.\n]?\\س*تحبي[^\n]{0,12}(اسجل|افكر|اضيف|نكمل|اسجل)[^!\n]{0,45}[?]\س*[🥰❤️👉]*\س*$\u;
159
160 // Matches a customer objection/hesitation ("هفكر"، "غالي"... ) – see
161 // stripRepeatedCta below.
162 const OBJECTION_RE = /([عاليه|هفكرها|هفكر|متأكد|مش متأكد|مش حاسمة|مش قادرة|مش قادر])too expensive|think about it)/i;
163
164 // Safety net: rewriting this prompt rule several times ("only ask once")
165 // never fully stopped the model from re-asking "تحبي تضيفيه لأوردرك؟" after
166 // almost every reply – trusting the model's own restraint here proved
167 // unreliable in real conversations. Enforce it deterministically instead:
168 // scan this session's own history (not the model's memory of its own
169 // rules) for a prior occurrence, and strip the nudge if it already fired.
170 //
171 // Exception: if the customer raised an objection ("هفكر"/"غالي") *after*
172 // that prior nudge, this new one is a fresh close attempt following the
173 // bot's answer to that objection – not a repeat of the same ask – so it's
174 // allowed through. Blanket-suppressing here would silence the bot at
175 // exactly the moment right after resolving a doubt, which is the most
176 // natural point to actually ask for the close.
177 function stripRepeatedCta(replyText, chatHistory) {
178   if (!CTA_NUDGE_RE.test(replyText)) return replyText;
179   const history = chatHistory || [];
180   let lastCtaIdx = -1;
181   for (let i = history.length - 1; i >= 0; i--) {
182     if (history[i].role === 'model' && CTA_NUDGE_RE.test(history[i].text || '')) {
183       lastCtaIdx = i;
184       break;
185     }
186   }
187   if (lastCtaIdx === -1) return replyText;
188   const objectionSince = history
189     .slice(lastCtaIdx + 1)
190     .some((h) => h.role === 'user' && OBJECTION_RE.test(h.text || ''));
191   if (objectionSince) return replyText;
192   return replyText.replace(CTA_NUDGE_RE, '').trim();
193 }
194
195 // Diagnostic trail only – when a checkout-flow regex below doesn't
196 // recognize the customer's message, log it (visible in Vercel's function
197 // logs) instead of silently falling through. Lets us later search real
198 // conversations for phrasings our patterns are missing, instead of
199 // guessing which ones to add.
200 function logUnmatchedIntent(context, sid, message) {
201   console.warn('[DIAG unmatched-intent]', JSON.stringify({ context, sid, message }));
202 }
203
204 // Returns the customer's past purchased product names by phone, so a
205 // returning customer's session can open with "X فكرة إلك جربت" instead of
206 // starting from zero every time. Uses the service-role client only –
207 // see get_customer_past_products' grants in migration 075.
208 async function getPastProducts(phone) {
209   if (!sbService || !phone) return [];
210   try {
211     const { data, error } = await sbService.rpc('get_customer_past_products', { p_phone: phone });
212     if (error || !data?.found) return [];
213     return data.products || [];
214   } catch {
215     return [];
216   }
217 }
218
219 // Sums a customer's earned loyalty points across every order tied to her
220 // phone number (guest checkout creates a fresh customers row per order –
221 // no shared row to read a single balance off, see migration 078's comment).
222 // service_role only, same access level as getPastProducts above.
223 async function getPointsBalance(phone) {
224   const normalized = normalizePhone(phone);
225   if (!sbService || !normalized) return null;
226   try {
227     const { data, error } = await sbService.rpc('get_customer_points_by_phone', { p_phone: normalized });
228     if (error) return null;
229     return data?.points ?? 0;
230   } catch {
231     return null;
232   }
233 }
234
235 // sid convention: web sessions look like "sid_xxxxx" (from
236 // chat-widget.js's localStorage id); other channels are prefixed
```

```

237 // "whatsapp:", "messenger:", "Instagram:" by their webhook handler.
238 function channelOf(sid) {
239   const idx = sid.indexOf(':');
240   return idx === -1 ? 'web' : sid.slice(0, idx);
241 }
242
243 const tools = [{
244   functionDeclarations: [
245     {
246       name: 'list_products',
247       description: 'يرجع قائمة المنتجات الحقيقية والحالية بأسعارها من قاعدة البيانات. استخدمها دائما قبل ذكر أي سعر أو اسم منتج.',
248       parameters: { type: SchemaType.OBJECT, properties: {} }
249     },
250     {
251       name: 'list_governorates',
252       description: 'يرجع أسماء كل المحافظات المتاحة للشحن مع سعر الشحن لكل واحدة.',
253       parameters: { type: SchemaType.OBJECT, properties: {} }
254     },
255     {
256       name: 'add_to_cart',
257       description: 'يضيف منتج للسلة الحقيقية (أو يحدث الكمية لو موجود بالفعل). استخدمها فورًا أول ما العميل يذكر منتج عايزه.',
258       parameters: {
259         type: SchemaType.OBJECT,
260         properties: {
261           product_id: { type: SchemaType.NUMBER, description: 'رقم المنتج من list_products' },
262           qty: { type: SchemaType.NUMBER, description: 'الكمية المطلوبة' }
263         },
264         required: ['product_id', 'qty']
265       }
266     },
267     {
268       name: 'remove_from_cart',
269       description: 'يشيل منتج من السلة.',
270       parameters: {
271         type: SchemaType.OBJECT,
272         properties: { product_id: { type: SchemaType.NUMBER } },
273         required: ['product_id']
274       }
275     },
276     {
277       name: 'view_cart',
278       description: 'عشان تأكدي للعميل الطلب show_checkout_form يرجع محتوى السلة الحالي بالتفصيل (أسماء حقيقية وأسعار وكميات وإجمالي). استخدمها قبل.',
279       parameters: { type: SchemaType.OBJECT, properties: {} }
280     },
281     {
282       name: 'show_checkout_form',
283       description: 'تفتح فورم حقيقي على الموقع ياخذ فيه العميل بياناته (اسم/موبايل/عنوان/محافظة) ويأكد الطلب بنفسه. استخدمها بس لو القناة هي الموقع - لو مش الموقع، النظام بيعت لينك المتجر.',
284       parameters: { type: SchemaType.OBJECT, properties: {} }
285     },
286     {
287       name: 'get_points_balance',
288       description: 'يرجع رصيد نقاط برنامج المكافآت الحقيقي لعملية معينة برقم موبايلها. استخدمها لو سألت عن رصيد نقاطها - لو رقمها مش معروف من كلامها في المحادثة، أسألها عنه الأول.',
289       parameters: {
290         type: SchemaType.OBJECT,
291         properties: { phone: { type: SchemaType.STRING, description: 'رقم موبايل العميل (11 رقم يبدأ ب 01)' } },
292         required: ['phone']
293       }
294     }
295   ]
296 }];
297
298 async function list_products() {
299   const { data, error } = await sb.from('products')
300     .select('id, name, price, old_price, size, slug, description, ingredients, skin_type, stock, rating, review_count')
301     .eq('is_active', true)
302     .order('sort_order');
303   if (error) return { error: error.message };
304   return { products: data };
305 }
306
307 // Used whenever the model itself couldn't produce a real answer - either
308 // it returned nothing (empty response) or the Gemini API call threw
309 // (503 overload, rate limit, network blip). Either way the customer asked
310 // something (often a vague price question like "بكام"/"hm"), so showing
311 // real prices beats silence or a bare "error, try again" that ManyChat's
312 // Response Mapping has no `reply` field to send at all.
313 async function productListFallbackReply(prefix) {
314   const { products } = await list_products();
315   const list = (products || []).map(p => `• ${p.name} - ${Math.round(p.price)} جنيه`).join('\n');
316   return `${prefix}قوليلي تحبي تعرفي إيه عن أي منتج منهم:\n\n${list}\n\nمتجانتا المتاحة: ${prefix}`;
317 }
318
319 async function list_governorates() {
320   const { data, error } = await sb.from('shipping_rates')
321     .select('governorate, cost')
322     .eq('is_active', true)
323     .order('sort_order');

```

```

324 if (error) return { error: error.message };
325 return { governorates: data };
326 }
327
328 async function add_to_cart(args, sid) {
329   const productId = Number(args.product_id);
330   const qty = Math.max(1, Math.floor(Number(args.qty))) || 1;
331   const { data: p, error } = await sb.from('products').select('id, name, price, image_url, stock').eq('id', productId).single();
332   if (error || !p) return { error: 'صحيح id عشان تاخدي list_products منتج غير معروف - استخدمني' };
333   if ((p.stock ?? 0) <= 0) return { error: 'المنتج نفذ من المخزون حالياً - رشي منتج تاني بديل' };
334   const cappedQty = Math.min(qty, p.stock);
335
336   const cart = sessionCarts[sid] || (sessionCarts[sid] = []);
337   const existing = cart.find(i => i.id === p.id);
338   if (existing) existing.qty = cappedQty;
339   else cart.push({ id: p.id, name: p.name, image: p.image_url, price: p.price, qty: cappedQty });
340
341   return view_cart(sid);
342 }
343
344 async function remove_from_cart(args, sid) {
345   const productId = Number(args.product_id);
346   const cart = sessionCarts[sid] || [];
347   if (!cart.length) {
348     return {
349       error: 'cart_empty',
350       message: 'محتاج رقم الموبايل (MON-xxxxx) الأوردر فاضي - لو عاوز تحلل طلب اسجل',
351     };
352   }
353   sessionCarts[sid] = cart.filter(i => i.id !== productId);
354   return view_cart(sid);
355 }
356
357 function view_cart(sid) {
358   const cart = sessionCarts[sid] || [];
359   const subtotal = cart.reduce((s, i) => s + i.price * i.qty, 0);
360   return { cart: cart.map(i => ({ id: i.id, name: i.name, price: i.price, qty: i.qty })), subtotal };
361 }
362
363 async function create_order(args, sid, channel) {
364   const { name, phone, address, governorate } = args;
365   const cart = sessionCarts[sid] || [];
366   if (!name || !phone || !address || !governorate) {
367     return { error: 'بيانات ناقصة - لازم الاسم والموبايل والعنوان والمحافظة' };
368   }
369   if (!cart.length) {
370     return { error: 'الأوردر فاضي - ارجعي للشات وضيبي منتج الأول' };
371   }
372
373   // Re-fetch every price/name from the real table by id right before
374   // committing the order - never trust cached values, even our own
375   // server-side cart's, in case a product changed since it was added.
376   const ids = cart.map(i => i.id);
377   const { data: products, error: pErr } = await sb.from('products').select('id, name, price, image_url').in('id', ids);
378   if (pErr || !products || !products.length) return { error: 'تَعَبَّر التحقق من المنتجات' };
379
380   const orderItems = [];
381   let subtotal = 0;
382   for (const it of cart) {
383     const p = products.find(x => x.id === it.id);
384     if (!p) return { error: `منتج غير معروف: ${it.id}` };
385     orderItems.push({ id: p.id, name: p.name, image: p.image_url, price: p.price, qty: it.qty });
386     subtotal += p.price * it.qty;
387   }
388
389   const { data: govMatch } = await sb.from('shipping_rates')
390     .select('governorate, cost')
391     .ilike('governorate', `${governorate}%`)
392     .eq('is_active', true)
393     .limit(1)
394     .single();
395   if (!govMatch) return { error: 'وتختاري اسم من القائمة بالظبط list_governorates المحافظة دي مش متاحة، لازم تستخدمني' };
396
397   const shipping = govMatch.cost;
398   const total = subtotal + shipping;
399   const deposit = 200;
400
401   const { data: order, error: oErr } = await sb.rpc('create_guest_order', {
402     p_customer: { name, phone, address, city: govMatch.governorate },
403     p_items: orderItems,
404     p_payment_method: 'cod',
405     p_delivery_method: 'standard',
406     p_subtotal: subtotal,
407     p_shipping_cost: shipping,
408     p_discount: 0,
409     p_total: total,
410     p_governorate: govMatch.governorate,
411     p_deposit_amount: deposit,

```

```

412 p_chat_session_id: sid,
413 // ManyChat's automations already send an explicit "messenger"/"instagram"
414 // hint with every request – channelOf(sid) alone can't tell them apart
415 // since ManyChat unifies both under the same "manychat:<contact id>"
416 // session prefix. Losing that distinction here (not just in the chat
417 // session record, which a prior fix already handles) meant the Telegram
418 // "confirm order" button could never tell notifyCustomerOrderConfirmed()
419 // an order's customer was actually on Instagram.
420 p_chat_channel: ['messenger', 'instagram', 'whatsapp'].includes(channel) ? channel : channelOf(sid),
421 });
422 if (oErr) return { error: 'حصلت مشكلة أثناء إنشاء الأوردر، حاول تاني' };
423
424 let transferNumber = '', transferName = '';
425 try {
426   const { data: n } = await sb.from('site_settings').select('value').eq('key', 'instapay_wallet_number').single();
427   const { data: nm } = await sb.from('site_settings').select('value').eq('key', 'instapay_wallet_name').single();
428   transferNumber = n?.value || ''; transferName = nm?.value || '';
429 } catch (e) { /* bot can still confirm the order without the transfer number handy */ }
430
431 sessionCarts[sid] = []; // order placed – start the next one empty
432
433 return {
434   order_number: order?.order?.order_number || order?.order_number,
435   phone,
436   customer_name: name,
437   total,
438   deposit_amount: deposit,
439   transfer_number: transferNumber,
440   transfer_name: transferName
441 };
442 }
443
444 function show_checkout_form(sid) {
445   return { action: 'form_shown', ...view_cart(sid) };
446 }
447
448 // Used by tryExplicitAddToCart below to match an explicit product mention
449 // (requires its own explicit "add to cart" phrase, unlike the removed
450 // autoDetectCartAdd which used to match on product-name substrings alone –
451 // that silently added a product to the customer's order from a bare price
452 // question or symptom description, with no purchase intent at all).
453 function bigrams(name) {
454   const words = name.split(/\s+/).filter(Boolean);
455   const pairs = [];
456   for (let i = 0; i < words.length - 1; i++) pairs.push(words[i] + ' ' + words[i + 1]);
457   return pairs;
458 }
459
460 // Cross-sell pairing, mirrors the "## البيع" prompt guidance – used only by
461 // the deterministic fast path below.
462 const CROSS_SELL_PAIRS = {
463   'غسول التفيتح': 'كريم التفيتح',
464   'كريم التفيتح': 'غسول التفيتح',
465   'غسول علاج حب الشباب للوجه': 'غسول التفيتح',
466 };
467
468 // "ضيفي غسول التفيتح للسلة" (e.g. "اسم منتج محدد + فعل إضافة صريح") kept
469 // getting intercepted by the LLM's own diagnostic-question instinct even
470 // after several prompt rewrites – the model just wouldn't reliably treat
471 // an explicit named-product command as an exception to the "ask about skin
472 // type first" rule. Handling this one narrow, unambiguous case in code
473 // (matching the existing "don't trust the model with the cart" pattern)
474 // makes it 100% reliable instead of hoping the prompt wins that tug of war.
475 async function tryExplicitAddToCart(message, sid) {
476   const text = message.trim();
477   // Requires an explicit cart-action phrase, not just any product mention –
478   // otherwise a plain price question ("غسول التفيتح بكام؟") would silently
479   // add it to the cart without the customer asking for that.
480   if (!/(للسلة|فالسلة|بالسلة|للاوردر|بالأوردر|ضيفي|اضيفي|حطيلي)/i.test(text)) return null;
481
482   const { data: products, error } = await sb.from('products').select('id, name, price, image_url, stock').eq('is_active', true);
483   if (error || !products) return null;
484
485   const matched = products.find((p) => bigrams(p.name).some((bg) => text.includes(bg)));
486   if (!matched) return null;
487
488   if ((matched.stock ?? 0) <= 0) {
489     return { reply: `ننفذ من المخزون حاليًا يا قندم 🥲 تحبي أرشحك بنديل؟ ${matched.name} للأسف` }, sessionId: sid };
490   }
491
492   const cart = sessionCarts[sid] || (sessionCarts[sid] = []);
493   if (!cart.find((i) => i.id === matched.id)) {
494     cart.push({ id: matched.id, name: matched.name, image: matched.image_url, price: matched.price, qty: 1 });
495   }
496
497   let crossSellLine = '';
498   const pairName = CROSS_SELL_PAIRS[matched.name];
499   if (pairName && !cart.find((i) => i.name === pairName)) {

```

```

500 const pairProduct = products.find((p) => p.name === pairName && (p.stock ?? 0) > 0);
501 if (pairProduct) crossSellLine = `\\n\\nمعاه في نفس الروتين **${pairProduct.name}** تحبي أضيف معاه؟`;
502 }
503
504 return {
505   reply: `جنيه أضفته لأوردرك ${Math.round(matched.price)} بسعر **${matched.name}** تمام يا فندم`,
506   sessionId: sid,
507 };
508 }
509
510 // Matches a bare price question with no product named in it at all –
511 // "عسول التفّيح بكام؟" – as opposed to "بكام؟", "سعرهم بكام؟", "السعر كام؟"
512 // (which names a product and is left to the model, which already handles
513 // "specific product + price" correctly). Anchored to the whole message so
514 // it only fires on a genuinely bare question, never a longer sentence that
515 // happens to contain "كام" elsewhere.
516 function isPriceQuestion(message) {
517   const t = String(message || '').trim();
518   if (!t || t.length > 40) return false;
519   return /^(?!\s)*$/i.test(t);
520 }
521
522 // Real bug (live customer transcript): a customer named two products in one
523 // message ("بسال عن كريم الليزر وكريم التصبغات"), then asked "سعرهم كام؟" in
524 // her next message – a plain pronoun referring back to what she'd just
525 // named. The model ignored her prior message entirely and asked "which
526 // product exactly?", even though she'd already answered that. Rather than
527 // hope a prompt rule makes the model check its own history reliably (it
528 // already has one for the "first message" case and still doesn't always
529 // follow it – see productListFallbackReply above), resolve this
530 // deterministically: match named products in her current message first,
531 // fall back to her immediately preceding message if this one is a bare
532 // pronoun, and only show the full price list if neither names anything.
533 async function tryPriceQuestion(message, sid, chatHistory) {
534   if (!isPriceQuestion(message)) return null;
535
536   const { data: products, error } = await sb.from('products')
537     .select('id, name, price, old_price, slug, rating, review_count, stock')
538     .eq('is_active', true);
539   if (error || !products || !products.length) return null;
540
541   let matched = resolveProductMentions(message, products);
542
543   if (!matched.length) {
544     for (let i = (chatHistory || []).length - 1; i >= 0; i--) {
545       if (chatHistory[i]?.role !== 'user') continue;
546       matched = resolveProductMentions(chatHistory[i].text || '', products);
547       break; // only the immediately preceding customer message counts
548     }
549   }
550
551   if (matched.length) {
552     const lines = matched.map((p) => {
553       const offer = p.old_price && p.old_price > p.price
554         ? `جنيه (عرض لفترة محدودة!) ${Math.round(p.old_price)} جنيه بدلاً من ${Math.round(p.price)} سعره حالياً`
555         : `جنيه ${Math.round(p.price)} سعره`;
556       const avail = (p.stock ?? 0) <= 0 ? 'غير متوفر حالياً' : '';
557       return `**${p.name}**\\n• ${offer}${avail}`;
558     });
559     return {
560       reply: `${lines.join('\\n\\n')}\\n\\nتحبي أضيفه لأوردرك يا فندم؟`,
561       sessionId: sid,
562     };
563   }
564
565   const greet = (chatHistory || []).length === 0 ? 'أهلاً بيكي يا فندم' : '';
566   return { reply: await productListFallbackReply(greet), sessionId: sid };
567 }
568
569 // Real bug (live customer transcript): "عندي تصبغت في وشي عايزة حاجة"
570 // "نقيد" names one specific, unambiguous skin concern that maps to exactly
571 // one real product (كريم التفّيح) – yet the model still asked "نوع بشرتك"
572 // "نوع بشرتك" instead of recommending it directly, even after the prompt was
573 // tightened to say diagnostic questions are only for genuinely vague asks.
574 // Tightening the prompt text alone didn't reliably fix it (verified live –
575 // same lesson as the price questions above), so ground this deterministically
576 // using the same PRODUCT_ALIAS_RULES: when exactly one real product matches
577 // what she said and it isn't already in her cart, inject a system note into
578 // this turn only (never persisted to chatHistory, never sent on any other
579 // turn) telling the model plainly which product this maps to.
580 async function buildSymptomNote(message, sid) {
581   const { data: products, error } = await sb.from('products')
582     .select('id, name')
583     .eq('is_active', true);
584   if (error || !products || !products.length) return null;
585
586   const matched = resolveProductMentions(message, products);

```

[illegible]

```

673 }
674
675 if (isCheckoutConfirmYes(message)) {
676   delete checkoutWizards[sid];
677   const summary = formatCartSummary(sid);
678   if (channelOf(sid) === 'web') {
679     return {
680       reply: `املي بيانات التوصيل! 📦! تمام\n\n${summary}`,
681       sessionId: sid,
682       showOrderForm: show_checkout_form(sid),
683       clearConfirm: true,
684     };
685   }
686   return { ...sendCheckoutLink(sid), clearConfirm: true };
687 }
688
689 if (isCheckoutConfirmNo(message)) {
690   delete checkoutWizards[sid];
691   return {
692     reply: 'تمام يا فندم 💖 قولي تحب تصنيف ايه في اوردرك',
693     sessionId: sid,
694     clearConfirm: true,
695   };
696 }
697
698 // Anything else (a price question, a new product, "تفاصيل...") isn't a
699 // yes/no answer to "do you want to check out?" – forcing the cart
700 // summary back at the customer here used to bury real questions like
701 // ""بكام؟" under an unrelated re-ask (and that re-ask text could itself
702 // get silently stripped by stripRepeatedCta if a similar nudge already
703 // fired earlier, making it look like the bot ignored the customer
704 // entirely). Drop the pending confirm and let the real message reach
705 // the model instead.
706 logUnmatchedIntent('checkout_confirm', sid, message);
707 delete checkoutWizards[sid];
708 return null;
709 }
710
711 async function runFunctionCall(name, args, sid) {
712   if (name === 'list_products') return list_products();
713   if (name === 'list_governorates') return list_governorates();
714   if (name === 'add_to_cart') return add_to_cart(args, sid);
715   if (name === 'remove_from_cart') return remove_from_cart(args, sid);
716   if (name === 'view_cart') return view_cart(sid);
717   if (name === 'show_checkout_form') return show_checkout_form(sid);
718   if (name === 'get_points_balance') {
719     const points = await getPointsBalance(args?.phone);
720     if (points === null) return { error: 'رقم الموبايل مش صحيح أو حصلت مشكلة – اسألنيها تبعته ثاني' };
721     return { points };
722   }
723   return { error: 'unknown function' };
724 }
725
726 const SITE_URL = 'https://www.montana.com.eg';
727
728 // -----
729 // Non-web checkout hand-off (WhatsApp/Messenger/Instagram) – these channels
730 // have no HTML form to render, and collecting name/address/phone/governorate
731 // in plain chat (the old submit_checkout_info flow) turned out unreliable in
732 // real use (the model could stall in a loop re-asking the same question, or
733 // even fabricate a fake "order confirmed" reply with no real order behind
734 // it – see the fabricated-order-claim guard below). Sending the customer's
735 // own real cart, tied via her existing sid, straight to the site's own
736 // checkout form sidesteps all of that: the same real HTML form, validation,
737 // and receipt upload the website already uses. Deterministic and hardcoded
738 // (never model-generated), like every other "start of a consequential step"
739 // reply in this file.
740 // -----
741 function sendCheckoutLink(sid) {
742   const url = `${SITE_URL}/complete-order.html?sid=${encodeURIComponent(sid)}`;
743   return {
744     reply: `📦: \n${url}`,
745     sessionId: sid,
746   };
747 }
748
749 // -----
750 // Main entry point – used by api/chat.js (web) and every webhook.
751 // -----
752 async function handleInboundMessage({ sid, message, formSubmit, channel }) {
753   const stored = await loadSession(sid);
754   sessionCarts[sid] = (stored.cart && stored.cart.length) ? stored.cart : (sessionCarts[sid] || []);
755   if (stored.checkoutWizard) checkoutWizards[sid] = stored.checkoutWizard;
756   let orderWizard = stored.orderWizard || null;
757   let chatHistory = stored.history || [];
758   let customerPhone = stored.customerPhone || null;
759   let customerName = stored.customerName || null;
760

```

```

761 const persist = async (patch = {}) => {
762   if (patch.orderWizard !== undefined) orderWizard = patch.orderWizard;
763   if (patch.cart !== undefined) sessionCarts[sid] = patch.cart;
764   if (patch.checkoutWizard !== undefined) {
765     if (patch.checkoutWizard) checkoutWizards[sid] = patch.checkoutWizard;
766     else delete checkoutWizards[sid];
767   }
768   if (patch.history !== undefined) chatHistory = patch.history;
769   if (patch.customerPhone !== undefined) customerPhone = patch.customerPhone;
770   if (patch.customerName !== undefined) customerName = patch.customerName;
771   await saveSession(sid, {
772     cart: sessionCarts[sid] || [],
773     history: chatHistory,
774     customerPhone,
775     customerName,
776     checkoutWizard: checkoutWizards[sid] || null,
777     orderWizard,
778     chatChannel: channel || undefined,
779   });
780 };
781
782 const finish = async (payload, userMsg = message) => {
783   if (payload?.reply) {
784     payload.reply = sanitizeReply(payload.reply);
785     payload.reply = stripRepeatedCta(payload.reply, chatHistory);
786   }
787   if (userMsg && payload?.reply) {
788     chatHistory = appendHistory(chatHistory, userMsg, payload.reply.replace(/\*\*/g, ''));
789   }
790   await persist({ history: chatHistory, cart: sessionCarts[sid] || [] });
791   return payload;
792 };
793
794 if (formSubmit) {
795   const order = await create_order(formSubmit, sid, channel);
796   if (order.error) return finish({ reply: `في مشكلة: ${order.error}`, sessionId: sid });
797   customerPhone = formSubmit.phone;
798   customerName = formSubmit.name;
799   // create_order() runs outside the model's normal tool-calling loop for
800   // this branch (formSubmit comes straight from complete-order.html /
801   // the website's inline form, not a chat message) – so the Gemini
802   // conversation itself never learns the order happened. Left alone,
803   // the model kept treating the (already-cleared) cart as still pending
804   // in the next chat message on the same session. Record it in history
805   // and drop the live chat session so the next message rebuilds it
806   // fresh, aware the order is done.
807   delete chatSessions[sid];
808   chatHistory = appendHistory(
809     chatHistory,
810     '(النظام: العميل كمل بيانات التوصيل وتسجل الأوردر)',
811     `اتسجل بنجاح والأوردر (السلعة) بقى فاضلي دلوقتى. لو العميل كلمني تاني، ده يبقى أوردرد جديد تمامًا ${order.order_number} تمام، الأوردر`
812   );
813   await persist({ customerPhone, customerName, cart: [] });
814   return finish({
815     reply: `شامل مقتم حجز) جنيهه ${Math.round(order.total)} والإجمالي **${order.order_number}** تمام يا فندم! ✅ اتسجل طلبك برقم ${Math.round(order.deposit_amount)} (${order.transfer_name}) (${order.transfer_number}) حوّلني المقتم على. (جنيهه هيتخصم منه)
    وارفعني صورة الإيصال تحت وهنالكلك الحجز على
    طول ♥️`,
816     sessionId: sid,
817     awaitingProof: { order_number: order.order_number, phone: formSubmit.phone },
818     order: {
819       order_number: order.order_number,
820       total: order.total,
821       deposit_amount: order.deposit_amount,
822       transfer_number: order.transfer_number,
823       transfer_name: order.transfer_name,
824     },
825   });
826 }
827
828 if (!message) throw Object.assign(new Error('Message required'), { status: 400 });
829
830 if (!genAI) {
831   throw Object.assign(new Error('Chat service unavailable'), { status: 503 });
832 }
833
834 const orderManageResult = await tryOrderManageWizard(message, sid, orderWizard, persist);
835 if (orderManageResult) return finish(orderManageResult);
836
837 const orderContextResult = await tryOrderContextReply({
838   message,
839   sid,
840   cart: sessionCarts[sid] || [],
841   customerPhone,
842   chatHistory,
843 });
844 if (orderContextResult) {
845   if (orderContextResult.startOrderWizard) {
846     await persist({

```

```

847     orderWizard: { step: 'phone', phone: null, orderNumber: null, orders: null },
848   });
849   delete orderContextResult.startOrderWizard;
850 }
851 return finish(orderContextResult);
852 }
853
854 if (checkoutWizards[sid]) {
855   const hadConfirm = checkoutWizards[sid].step === 'await_confirm';
856   const confirmResult = await tryAdvanceCheckoutConfirm(sid, message);
857   if (confirmResult) {
858     if (confirmResult.clearConfirm) await persist({ checkoutWizard: null });
859     return finish(confirmResult);
860   }
861   // tryAdvanceCheckoutConfirm bowed out (unmatched message) and already
862   // cleared the in-memory wizard – persist that so the next message
863   // (possibly on a different serverless instance) doesn't reload the
864   // stale await_confirm state from storage and re-hijack it again.
865   if (hadConfirm && !checkoutWizards[sid]) await persist({ checkoutWizard: null });
866 }
867
868 const explicitAddResult = await tryExplicitAddToCart(message, sid);
869 if (explicitAddResult) {
870   await persist({ cart: sessionCarts[sid] || [] });
871   return finish(explicitAddResult);
872 }
873
874 const priceQuestionResult = await tryPriceQuestion(message, sid, chatHistory);
875 if (priceQuestionResult) return finish(priceQuestionResult);
876
877 const agents = ['لانيان', 'نوران', 'نور'];
878 const agentIndex = Math.floor(Date.now() / (2 * 60 * 60 * 1000)) % agents.length;
879 const agentName = agents[agentIndex];
880
881 if (!chatSessions[sid]) {
882   const model = genAI.getGenerativeModel({
883     // gemini-pro-latest resolves to Gemini 3.1 Pro, whose free-tier daily
884     // quota (250 requests/day) the store's real traffic started exceeding
885     // – every request past that cap 429s and the customer got a generic
886     // fallback instead of a real answer for the rest of the day.
887     // gemini-2.5-pro 404s outright on this key ("no longer available to
888     // new users" – Google AI Studio's usage dashboard still lists it with
889     // quota, but the API itself refuses it). gemini-2.5-flash has 10,000/
890     // day on this account (40x the old cap) and isn't blocked.
891     model: 'gemini-2.5-flash',
892     systemInstruction: { parts: [{ text: SYSTEM_PROMPT }] },
893     tools,
894     // The model's internal "thinking" step eats into the same output-
895     // token budget as the visible reply, so a generous ceiling avoids an
896     // empty final response on tool-calling turns.
897     generationConfig: { maxOutputTokens: 16384 },
898   });
899   const pastProducts = customerPhone ? await getPastProducts(customerPhone) : [];
900   const pastProductsNote = pastProducts.length ? `اشترى قبل كده - ${pastProducts.join('، ')} : ` : ` `;
901   const memoryIntro = customerName
902     ? [{ role: 'user', parts: [{ text: `(العميل) ${customerName} ${customerPhone ? ' - ' + customerPhone : ''} - جلسة سابقة ${pastProductsNote}` } ] }
903     ,
904     { role: 'model', parts: [{ text: pastProducts.length ? 'تمام، فكرة العميل ❤️ : تمام، فكرة العميل ومنتجاته اللي جربها قبل كده ❤️ : ' : 'تمام، فكرة العميل' } ] }
905     : [];
906   chatSessions[sid] = model.startChat({
907     history: [
908       { role: 'user', parts: [{ text: `اسمك "${agentName}"` } ] },
909       { role: 'model', parts: [{ text: `من فريق مونتاليا ${agentName} أنا` } ] },
910       ...memoryIntro,
911       ...historyToGemini(chatHistory),
912     ],
913   });
914 }
915
916 const skipCheckoutNudge =
917   isProductAddConfirmation(message) &&
918   wasAskingToAddToCart(lastModelMessage(chatHistory));
919
920 if (!skipCheckoutNudge) {
921   const thanksNudge = maybeNudgeCheckoutOnThanks(message, sid);
922   if (thanksNudge) {
923     if (thanksNudge.awaitConfirm) {
924       checkoutWizards[sid] = { step: 'await_confirm' };
925       await persist({ checkoutWizard: checkoutWizards[sid] });
926     }
927     return finish(thanksNudge);
928   }
929 }
930
931 const symptomNote = await buildSymptomNote(message, sid);
932 const modelInputMessage = symptomNote ? `${message}\n\n${symptomNote}` : message;
933
934 const chat = chatSessions[sid];

```

```

934 let result, calls;
935 try {
936   result = await chat.sendMessage(modelInputMessage);
937   calls = result.response.functionCalls();
938
939   // Gemini occasionally returns an empty response (finishReason STOP, no
940   // text, no function call) – a transient model quirk on some prompts, not
941   // a real "nothing to say". Retry a couple of times before giving up.
942   for (let retry = 0; retry < 2 && !calls?.length && !result.response.text(); retry++) {
943     result = await chat.sendMessage(modelInputMessage);
944     calls = result.response.functionCalls();
945   }
946 } catch (e) {
947   // The Gemini API call itself failing (503 overload, rate limit, network
948   // error – seen live: "This model is currently experiencing high
949   // demand") used to propagate uncaught all the way to api/chat.js's
950   // outer catch, which returns a bare 500 with no `reply` field at all –
951   // ManyChat's Response Mapping has nothing to send, so the customer got
952   // total silence instead of an answer. Her message is very often a
953   // price/product question, so fall back to the real price list instead.
954   console.error('[DIAG gemini-failed]', e.message);
955   return finish({ reply: await productListFallbackReply('معلش يا فندم، حصل تأخير بسيط 🙏'), sessionId: sid });
956 }
957
958 // Still nothing after retries – rather than leave the customer with an
959 // empty non-answer, fall back to real product data directly.
960 if (!calls?.length && !result.response.text()) {
961   return finish({ reply: await productListFallbackReply('يا فندم'), sessionId: sid });
962 }
963
964 let formTriggered = null;
965 let awaitingProof = null;
966
967 let guard = 0;
968 while (calls && calls.length && guard < 10) {
969   const responses = [];
970   for (const call of calls) {
971     const output = await runFunctionCall(call.name, call.args, sid);
972     if (call.name === 'show_checkout_form') formTriggered = output;
973     if (call.name === 'create_order' && output?.order_number && !output.error) {
974       awaitingProof = {
975         order_number: output.order_number,
976         phone: output.phone || call.args?.phone || customerPhone,
977         customer_name: output.customer_name || call.args?.name || customerName,
978       };
979     }
980     responses.push({ functionResponse: { name: call.name, response: output } });
981   }
982   try {
983     result = await chat.sendMessage(responses);
984     calls = result.response.functionCalls();
985   } catch (e) {
986     console.error('[DIAG gemini-failed-followup]', e.message);
987     return finish({ reply: await productListFallbackReply('معلش يا فندم، حصل تأخير بسيط 🙏'), sessionId: sid });
988   }
989   guard++;
990 }
991
992 if (formTriggered) {
993   if (!isExplicitCheckoutIntent(message) && !isCheckoutConfirmYes(message)) {
994     checkoutWizards[sid] = { step: 'await_confirm' };
995     await persist({ checkoutWizard: checkoutWizards[sid] });
996     const summary = formatCartSummary(sid);
997     // Never use the model's own free-form text here – no order exists
998     // yet at this point (that's exactly what this branch means), and
999     // the model has fabricated "order prepared, we'll call you soon"
1000     // -style claims in this exact spot before. Fixed, honest copy only.
1001     return finish({
1002       reply: `تمام يا فندم 💖\n\n${summary}\n\n**حاجة ثانية** ولا 🛒 **تكمّل الطلب**، تحبي
1003       sessionId: sid,
1004     });
1005   }
1006   delete checkoutWizards[sid];
1007   await persist({ checkoutWizard: null });
1008   if (channelOf(sid) === 'web') {
1009     const replyText = result.response.text() || 'تمام يا فندم، نواني وهرجعك بكل التفاصيل';
1010     return finish({ reply: replyText, sessionId: sid, showOrderForm: formTriggered });
1011   }
1012   return finish(sendCheckoutLink(sid));
1013 }
1014
1015 // Safety net: the model sometimes describes "opening a form"/"sending a
1016 // link" in prose without actually calling show_checkout_form (seen on
1017 // ManyChat when a customer volunteers her shipping info unprompted) –
1018 // there's no real form on non-web channels, so fall back to the real
1019 // checkout link instead of leaving the customer staring at a form/link
1020 // that was never actually sent.
1021 //

```

```

1022 // Real bug (live customer transcript, Mohamed Abed) with the OLD chat-based
1023 // info-collection flow: this used to match on the bare word "فورم"
1024 // appearing ANYWHERE in the model's reply, and the system prompt itself
1025 // discusses "الفرم" extensively – so the model legitimately said the word
1026 // constantly while reasoning about NOT using one, and every one of those
1027 // legitimate replies got silently discarded. Narrowed to only match an
1028 // actual first-person claim of having sent/opened one ("هفتحلك فورم",
1029 // "هيتحلك لينك"), not any passing mention of the word.
1030 if (channelOf(sid) !== 'web' && /(هفتحلك|هيتحلك|بعتلك|فتحتلك|حارسلك|مرسلتك)/i.test(result.response.text() || '')) {
1031   return finish(sendCheckoutLink(sid));
1032 }
1033
1034 const rawText = result.response.text();
1035 if (!rawText) {
1036   console.error('[DIAG empty-text]', JSON.stringify({
1037     guard,
1038     finishReason: result.response.candidates?.[0]?.finishReason,
1039     safetyRatings: result.response.candidates?.[0]?.safetyRatings,
1040     promptFeedback: result.response.promptFeedback,
1041   }));
1042 }
1043 let replyText = rawText || 'تمام يا فندم، ثواني وهرجعلك بكل التفاصيل';
1044
1045 // Real bug (live test, non-reproducible on retry but confirmed via direct
1046 // DB check – no order existed): the model can free-text a completely
1047 // convincing "your order is confirmed, number MON-xxxxx" reply without
1048 // ever going through create_order – the only path that actually creates a
1049 // real order is the web formSubmit branch above, which sets
1050 // `awaitingProof`. Non-web channels never create an order in-chat at all
1051 // now (see sendCheckoutLink) – any order claim there is by definition
1052 // fabricated. If a reply claims an order was just registered but this turn
1053 // never actually produced one, never let it reach the customer as if a
1054 // real order/deposit now exists.
1055 if (!awaitingProof && /(MON-[\da-Za-z]{3,}|اتسجل\s*(طلبك|أوردرك)|تم\s*(طلبك|أوردرك)|*(الطلب|الأوردر|طلبك|أوردرك))/i.test(replyText)) {
1056   console.error('[DIAG fabricated-order-claim]', JSON.stringify({ sid, replyText }));
1057   replyText = '✅ تمام يا فندم، بس عايزة أتأكد قبل ما أسجل: البيانات اللي بعثتها صح كده!';
1058 }
1059
1060 return finish({ reply: replyText, sessionId: sid, awaitingProof: awaitingProof || undefined });
1061 }
1062
1063 // Used by complete-order.html to show what's actually in a customer's cart
1064 // before she fills in her delivery info – same real cart create_order will
1065 // use, read straight from persistence (no live model turn needed).
1066 async function getCartForSid(sid) {
1067   const stored = await loadSession(sid);
1068   const cart = Array.isArray(stored.cart) ? stored.cart : [];
1069   const subtotal = cart.reduce((s, i) => s + i.price * i.qty, 0);
1070   return { cart, subtotal };
1071 }
1072
1073 module.exports = { handleInboundMessage, channelOf, chatSessions, sessionCarts, getCartForSid, getPointsBalance };
1074

```



```

83 // a product mid-conversation (that fuzzier case is chatEngine's job).
84 // Returns every match (not just the first) so the caller can ask the
85 // customer to pick when a short name like "كريم" matches more than one
86 // real product, instead of silently guessing.
87 async function matchProducts(text) {
88   const { data: products, error } = await sb.from('products').select('id, name, price, image_url').eq('is_active', true);
89   if (error || !products) return [];
90   const t = String(text || '').trim();
91   if (!t) return [];
92   const exact = products.filter(p => t.includes(p.name) || p.name.includes(t));
93   if (exact.length) return exact;
94   // Customer typed a colloquial name ("كريم اللبزر") instead of the exact
95   // catalog name ("كريم العناية بعد اللبزر") – same alias resolution the
96   // chat price-question fast path uses, so this step doesn't wrongly tell
97   // her the product doesn't exist.
98   return resolveProductMentions(t, products);
99 }
100
101 function formatOrdersList(orders) {
102   return orders.map((o, i) =>
103     `${i + 1}. \`${o.order_number}\` - ${statusAr(o.status)} - ${Math.round(o.total)} ج.م`
104   ).join('\n');
105 }
106
107 function pickOrderFromMessage(message, orders) {
108   const t = String(message || '').trim();
109   const mon = t.match(/MON-\d{5}/i);
110   if (mon) return mon[0].toUpperCase();
111   const num = parseInt(toWesternDigits(t), 10);
112   if (num >= 1 && num <= orders.length) return orders[num - 1].order_number;
113   return null;
114 }
115
116 /**
117  * @returns {object|null} response payload or null to continue normal chat
118  */
119 async function tryOrderManageWizard(message, sid, wizard, persist) {
120   let w = wizard;
121
122   if (!w && wantsOrderManagement(message)) {
123     w = { step: 'phone', phone: null, orderNumber: null, orders: null };
124     await persist({ orderWizard: w });
125     return {
126       reply: 'تمام يا قندم ❤ عشان الغي أو أعدّل الطلب، ابعتيلي **رقم الموبايل** اللي سجّلتيه في الطلب (11 رقم)',
127       sessionId: sid,
128     };
129   }
130
131   if (!w) return null;
132
133   const text = String(message || '').trim();
134
135   if (w.step === 'phone') {
136     const phone = normalizePhone(text);
137     if (!phone) {
138       // Real bug (live customer transcript): a customer accidentally in
139       // this wizard kept asking an unrelated question ("موجودين في صيدليات")
140       // and got the same "invalid phone number" error on every single
141       // reply, forever – the step never let a message through unless it
142       // happened to parse as a phone number. If her text has no digits at
143       // all, she's clearly not attempting one – drop the wizard and let the
144       // real message reach normal handling instead of trapping her here.
145       const hasDigits = /[dⁱ-ⁱ-ⁱ]/.test(text);
146       if (!hasDigits) {
147         await persist({ orderWizard: null });
148         return null;
149       }
150       return { reply: 'رقم الموبايل مش صحيح – لازم يبدأ ب 01 ويكون 11 رقم 📞', sessionId: sid };
151     }
152     const listed = await listOrders(phone);
153     if (listed.error) {
154       await persist({ orderWizard: null });
155       return { reply: 'حصلت مشكلة، حاولي تاني 🙄', sessionId: sid };
156     }
157     if (!listed.found) {
158       await persist({ orderWizard: null });
159       return { reply: 'MON-xxxxx مفيش طلبات نشطة على الرقم ده. تأكدي من الرقم أو ابعتيلي رقم الطلب 📞', sessionId: sid };
160     }
161     const orders = listed.orders;
162     w = { step: orders.length === 1 ? 'action' : 'pick_order', phone, orders, orderNumber: orders.length === 1 ? orders[0].order_number :
163 null };
164     await persist({ orderWizard: w, customerPhone: phone });
165
166     if (orders.length === 1) {
167       const o = orders[0];
168       return {
169         reply: `تعديل\ng• إلغاء الطلب\ng• اختاري\ng• \`${o.order_number}\` - ${statusAr(o.status)} - ${Math.round(o.total)} ج.م`
170         , `خروج` - إلغاء العملية\ng• إضافة منتج` - تصنيفي منتج على نفس الطلب\ng• عنوان` - تغيير العنوان\ng• المحافظة

```

```

170     });
171   }
172   return {
173     reply: `الرقم من القائمة (1, 2...) (MON-xxxxx) *ابعتلي رقم الطلب*\n\n${formatOrdersList(orders)}\n\nطلبات على الرقم ده ${orders.length} لقيت`,
174     sessionId: sid,
175   };
176 }
177
178 if (w.step === 'pick_order') {
179   const picked = pickOrderFromMessage(text, w.orders || []);
180   if (!picked) {
181     return { reply: 'أر رقم من القائمة MON-xxxxx معلش، مش فاهمة. ابعتلي رقم الطلب', sessionId: sid };
182   }
183   w.step = 'action';
184   w.orderNumber = picked;
185   await persist({ orderWizard: w });
186   return {
187     reply: `إضافة منتج - تضيفي منتج على نفس الطلب *n* تعديل عنوان - تغيير العنوان/المحافظة *n* الغاء - إلغاء الطلب *n* اختاري: \n\n${picked} \n\n تمام - الطلب`,
188     sessionId: sid,
189   };
190 }
191
192 if (w.step === 'action') {
193   if (/^(\s|cancel|خروج|إلغاء)/i.test(text)) {
194     await persist({ orderWizard: null });
195     return { reply: 'تمام، تلغت العملية 😊 تحبي حاجة ثانية؟', sessionId: sid };
196   }
197   if (/^(\s|cancel|إلغاء|إلغاء|إلغاء)/i.test(text)) {
198     const res = await cancelOrder(w.orderNumber, w.phone);
199     await persist({ orderWizard: null });
200     if (res.error === 'not_found') return { reply: 'الطلب مش موجود على الرقم ده', sessionId: sid };
201     if (res.error === 'cannot_cancel') return { reply: `مش ممكن تلغي الطلب - حالته: ${statusAr(res.status)} 🚫`, sessionId: sid };
202     if (!res.ok) return { reply: 'تعثّر الإلغاء، حاولي تاني أو تواصلني معنا', sessionId: sid };
203     if (res.already) return { reply: `ملغي بالفعل ✅ \n\n${w.orderNumber} \n\n الطلب`, sessionId: sid };
204     return { reply: `بنجاح ✅ \n\n${w.orderNumber} \n\n تم إلغاء الطلب`, sessionId: sid };
205   }
206   if (/^(modify|تعديل|تعديل عنوان|عدل|عدل العنوان)/i.test(text)) {
207     w.step = 'new_address';
208     await persist({ orderWizard: w });
209     return { reply: 'أبعتلي العنوان الجديد بالتفصيل (المنطقة + الشارع)', sessionId: sid };
210   }
211   if (/^(اضيف|أضيف|إضافة|إضافة)/i.test(text)) {
212     w.step = 'add_product';
213     await persist({ orderWizard: w });
214     return { reply: 'تمام، عايزة تضيفي إيه؟ اكتبني اسم المنتج', sessionId: sid };
215   }
216   return { reply: '**اختاري: الغاء** أو **تعديل عنوان** أو **إضافة منتج** أو **خروج**', sessionId: sid };
217 }
218
219 if (w.step === 'add_product') {
220   const matches = await matchProducts(text);
221   if (!matches.length) {
222     return { reply: 'أعلاه، مش لاقياه. اكتبني اسم المنتج زي ما هو في المتجر', sessionId: sid };
223   }
224   if (matches.length > 1) {
225     const names = matches.map((p) => `• ${p.name}`).join('\n');
226     return {
227       reply: `في \n\n${names} أكثر من منتج قريب من كده، أي واحد بالطيط؟`,
228       sessionId: sid,
229     };
230   }
231   const product = matches[0];
232   w.pendingProduct = product;
233   w.step = 'add_quantity';
234   await persist({ orderWizard: w });
235   return { reply: `كلم قطعة عايزة؟ - ${product.name}`, sessionId: sid };
236 }
237
238 if (w.step === 'add_quantity') {
239   const digits = toWesternDigits(text).replace(/^[^d]/g, '');
240   let qty = parseInt(digits, 10);
241   if (!qty || qty < 1) {
242     if (/^(واحد|واحدة|قطعة|علبة)/i.test(text)) qty = 1;
243     else return { reply: 'أكتبني رقم صحيح للكمية', sessionId: sid };
244   }
245   const result = await addItemToOrder(w.orderNumber, w.phone, w.pendingProduct.id, qty);
246   await persist({ orderWizard: null });
247   if (result.error === 'window_expired') {
248     return { reply: 'معلش، عتّى أكثر من 6 ساعات على الطلب فمّش ينفع نضيف عليه - لازم تعملي طلب جديد منفصل', sessionId: sid };
249   }
250   if (result.error === 'cannot_modify') {
251     return { reply: `مش ممكن نضيف على الطلب ده - حالته: ${statusAr(result.status)} 🚫`, sessionId: sid };
252   }
253   if (result.error === 'out_of_stock') {
254     return { reply: 'معلش، الكمية دي مش متاحة دلوقتي 🙏', sessionId: sid };
255   }
256 }

```

```

256   if (result.error === 'not_found') {
257     return { reply: 'الطلب مش موجود على الرقم ده', sessionId: sid };
258   }
259   if (!result.ok) {
260     return { reply: 'تَعذّر الإضافة، حاولي ثاني أو تواصلني معانا', sessionId: sid };
261   }
262   return {
263     reply: `الإجمالي بقي \`${result.order_number}\` لطلبك \`${result.added_product} × ${result.added_qty} تمام! ✅ اتضاف`,
264     ${Math.round(result.new_total)} ج.م,
265     sessionId: sid,
266   };
267 }
268
269 if (w.step === 'new_address') {
270   if (text.length < 5) return { reply: 'العنوان قصير - اكتبيه بالتفصيل أكثر', sessionId: sid };
271   w.newAddress = text;
272   w.step = 'new_governorate';
273   await persist({ orderWizard: w });
274   return { reply: 'تمام - **المحافظة** الجديدة?', sessionId: sid };
275 }
276
277 if (w.step === 'new_governorate') {
278   const res = await updateDelivery(w.orderNumber, w.phone, {
279     address: w.newAddress,
280     governorate: text,
281   });
282   await persist({ orderWizard: null });
283   if (res.error === 'not_found') return { reply: 'الطلب مش موجود', sessionId: sid };
284   if (res.error === 'cannot_modify') return { reply: `مش ممكن نعدل - الحالة: ${statusAr(res.status)}`, sessionId: sid };
285   if (res.error === 'invalid_governorate') return { reply: 'اكتبي اسم محافظة من اللي عندنا', sessionId: sid };
286   if (!res.ok) return { reply: 'تَعذّر التعديل، حاولي ثاني', sessionId: sid };
287   return {
288     reply: `ج.م: ${Math.round(res.total)} الإجمالي: 🏠 ${res.governorate} 📍 ${res.address} 📞 📌 \`${res.order_number}\` تم تحديث الطلب ✅\n`,
289     sessionId: sid,
290   };
291 }
292
293 return null;
294 }
295
296 module.exports = {
297   wantsOrderManagement,
298   tryOrderManageWizard,
299   normalizePhone,
300 };

```

lib/chatSessionStore.js

```
1 // Persist chat cart, conversation history, and wizard state in Supabase.
2 const { createClient } = require('@supabase/supabase-js');
3
4 const SUPABASE_URL = process.env.SUPABASE_URL || 'https://ikryeyqrithikabwidov.supabase.co';
5 const SUPABASE_KEY = process.env.SUPABASE_ANON_KEY ||
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImIrcnllcXFyaXRoaWthYndpZG92Iiwicm9sZSI6ImFub24iLCJpYXQiOiE3ODk0MzY1ODcsImV4cCI6IjE5MjMxMjU4N30.kNfHPxV4fq67cOF8uFsTrL0xPYcLcmmD03ScIHzi9Uo';
6
7 const sb = createClient(SUPABASE_URL, SUPABASE_KEY);
8 const MAX_HISTORY = 24;
9
10 async function loadSession(sid) {
11   const { data, error } = await sb.rpc('get_chat_bot_session', { p_session_id: sid });
12   if (error || !data?.found) {
13     return {
14       cart: [],
15       history: [],
16       customerPhone: null,
17       customerName: null,
18       checkoutWizard: null,
19       orderWizard: null,
20     };
21   }
22   return {
23     cart: Array.isArray(data.cart) ? data.cart : [],
24     history: Array.isArray(data.history) ? data.history : [],
25     customerPhone: data.customer_phone || null,
26     customerName: data.customer_name || null,
27     checkoutWizard: data.checkout_wizard || null,
28     orderWizard: data.order_wizard || null,
29   };
30 }
31
32 async function saveSession(sid, state) {
33   if (!sid) return;
34   const history = (state.history || []).slice(-MAX_HISTORY);
35   await sb.rpc('save_chat_bot_session', {
36     p_session_id: sid,
37     p_cart: state.cart ?? null,
38     p_history: history,
39     p_customer_phone: state.customerPhone ?? null,
40     p_customer_name: state.customerName ?? null,
41     p_checkout_wizard: state.checkoutWizard === undefined ? null : state.checkoutWizard,
42     p_order_wizard: state.orderWizard === undefined ? null : state.orderWizard,
43     p_chat_channel: state.chatChannel ?? null,
44   });
45 }
46
47 function appendHistory(history, userText, modelText) {
48   const next = [...(history || [])];
49   if (userText) next.push({ role: 'user', text: String(userText).slice(0, 2000) });
50   if (modelText) next.push({ role: 'model', text: String(modelText).slice(0, 4000) });
51   return next.slice(-MAX_HISTORY);
52 }
53
54 function historyToGemini(history) {
55   return (history || []).map((h) => ({
56     role: h.role === 'model' ? 'model' : 'user',
57     parts: [{ text: h.text }],
58   }));
59 }
60
61 // Arabic-Indic (٠-٩) and Extended Arabic-Indic/Persian (۰-۹) digits are
62 // common on Arabic keyboards but don't match JS's \d - convert them to
63 // Western digits first so phone/number input isn't silently rejected.
64 function toWesternDigits(str) {
65   return String(str || '').replace(/[\u{1d160}-\u{1d16f}]/g, (ch) => {
66     const code = ch.charCodeAt(0);
67     if (code >= 0x0660 && code <= 0x0669) return String(code - 0x0660);
68     if (code >= 0x06F0 && code <= 0x06F9) return String(code - 0x06F0);
69     return ch;
70   });
71 }
72
73 function normalizePhone(text) {
74   const digits = toWesternDigits(text).replace(/[^0-9]/g, '');
75   if (/^01\d{9}$/.test(digits)) return digits;
76   return null;
77 }
78
79 module.exports = {
80   loadSession,
81   saveSession,
82   appendHistory,
```

```
83 historyToGemini,  
84 normalizePhone,  
85 toWesternDigits,  
86 MAX_HISTORY,  
87 };  
88
```

```

1 // Thin Vercel function wrapper – the real logic lives in lib/chatEngine.js.
2 const { handleInboundMessage, channelOf, getCartForSid, getPointsBalance } = require('../lib/chatEngine');
3 const { enforceRateLimit, rateLimit, hasValidApiSecret } = require('../lib/security');
4
5 // Trusted external senders (e.g. ManyChat relaying Facebook/Instagram DMs and
6 // comments to this same bot brain) authenticate via the X-Montana-Secret header
7 // and get rate-limited per conversation instead of per IP – those platforms'
8 // outbound requests can share IPs across many unrelated businesses, so an
9 // IP-keyed limit would risk throttling real customers.
10 module.exports = async (req, res) => {
11   if (req.method !== 'POST') return res.status(405).json({ error: 'POST only' });
12
13   try {
14     const trusted = hasValidApiSecret(req);
15     const { message, sessionId, formSubmit, channel, getCart, getPoints, phone } = req.body || {};
16
17     // Lightweight read used by complete-order.html to show the customer's
18     // real cart when she arrives via a checkout link from a non-web channel
19     // – no model turn, just the persisted cart for this sid.
20     if (getCart) {
21       const sid = String(sessionId || '').slice(0, 64);
22       if (!sid) return res.status(400).json({ error: 'sessionId required' });
23       enforceRateLimit(req, 'chat-getcart', { windowMs: 60_000, max: 60 });
24       const { cart, subtotal } = await getCartForSid(sid);
25       return res.json({ cart, subtotal });
26     }
27
28     // Lightweight read used by complete-order.html to show the loyalty
29     // points balance for whatever phone number the customer has typed so
30     // far into the form.
31     if (getPoints) {
32       enforceRateLimit(req, 'chat-getpoints', { windowMs: 60_000, max: 30 });
33       const points = await getPointsBalance(phone);
34       return res.json({ points: points ?? 0 });
35     }
36
37     if (!message && !formSubmit) {
38       return res.status(400).json({ error: 'message required' });
39     }
40     if (message && String(message).length > 2000) {
41       return res.status(400).json({ error: 'message too long' });
42     }
43     const sid = String(sessionId || 'default').slice(0, 64);
44     const safeChannel = ['messenger', 'instagram'].includes(channel) ? channel : undefined;
45     if (trusted) {
46       rateLimit(`chat:external:${sid}`, { windowMs: 60_000, max: 40 });
47     } else {
48       enforceRateLimit(req, 'chat', { windowMs: 60_000, max: 40 });
49     }
50     const payload = await handleInboundMessage({ sid, message, formSubmit, channel: safeChannel });
51     // ManyChat (messenger/instagram) can't split a single field into
52     // multiple bubbles itself – its "Send Message" steps each need their
53     // own variable. Split the reply into up to 3 parts here so the
54     // ManyChat flow can send them as separate messages; reply1 always
55     // holds the first (or only) part so it still works if nothing maps
56     // reply2/reply3.
57     if (payload?.reply) {
58       // Facebook/Instagram/WhatsApp render Send Message text as plain text –
59       // unlike chat-widget.js's formatReply(), they don't turn bold into
60       // real bold, so the asterisks would show up literally to the customer.
61       // The website widget still gets the raw markdown (it renders it).
62       if (channelOf(sid) !== 'web') {
63         payload.reply = payload.reply.replace(/\\*(.?)\\*/g, '$1');
64       }
65       // "NONE" instead of "" – ManyChat's "has any value" condition treated
66       // an empty string inconsistently (a real customer got the literal
67       // unresolved "{{Bot Reply 2}}" text sent to them once the field was
68       // empty). An explicit sentinel the Condition step can string-match
69       // against ("isn't NONE") removes that ambiguity entirely.
70       let parts = String(payload.reply).split(/\\s*\\n+/).map(p => p.trim()).filter(Boolean);
71       // Fallback: chatEngine.js's sanitizeReply() already normalizes the
72       // known bullet-list case into blank-line-separated paragraphs, but if
73       // the model returns one long block some other way (no blank lines at
74       // all), force a split on single newlines instead of leaving the
75       // customer with one giant unreadable bubble.
76       if (parts.length === 1 && parts[0].length > 500) {
77         const lines = parts[0].split('\\n').map(l => l.trim()).filter(Boolean);
78         if (lines.length > 1) parts = lines;
79       }
80       payload.reply1 = parts[0] || payload.reply;
81       payload.reply2 = parts.length > 1 ? parts[1] : 'NONE';
82       payload.reply3 = parts.length > 2 ? parts.slice(2).join('\\n\\n') : 'NONE';
83     }
84     res.json(payload);

```

```
85 } catch (error) {
86   if (error.status === 400) return res.status(400).json({ error: error.message });
87   if (error.status === 429) return res.status(429).json({ error: 'Too many requests' });
88   console.error('Chat error:', error.message);
89   res.status(500).json({ error: 'حصل مشكلة، حاول ثاني' });
90 }
91 };
92
```